Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО

Факультет инженерии искусственного интеллекта

Дисциплина:
«Архитектура вычислительных систем»

ПРАКТИЧЕСКАЯ РАБОТА № 2

«Очистка данных и удалённая классификация на двух узлах»

Выполнил:
Степанов Егор Андреевич, студент группы J3124

Проверил:
Карепин Денис Дмитриевич

Санкт-Петербург
2026 г.

СОДЕРЖАНИЕ

Цель работы.....3
Состав проекта и описание скриптов.....3
1. Настройка SSH на обоих хостах.....5
2. Проверка сети и Docker.....7
3. Сборка и запуск контейнеров.....8
4. Подготовка и очистка данных на хосте А.....10
5. Передача очищенного файла на хост Б.....11
6. Обучение модели на хосте Б.....11
7. Запуск Flask-сервиса и запрос предсказаний.....12
8. Ограничение доступа к порту 5000.....14
Вывод.....16
Использование GAI.....17

Цель работы

Цель работы — освоить распределённую обработку данных на двух Linux-хостах: подготовить и очистить табличный набор данных в Docker-контейнере на первом узле, безопасно передать его по SSH/SCP на второй узел, обучить там модель классификации, развернуть Flask-сервис для получения предсказаний по сети и ограничить доступ к нему средствами межсетевого экрана iptables.

Стенд состоит из двух физических компьютеров под управлением Arch Linux, подключённых к одной сети Wi-Fi:

– **хост А** (узел подготовки данных) — интерфейс `wlp0s20f3`, адрес `192.168.0.108`, контейнер `data-processor`;

– **хост Б** (узел обучения и сервиса) — интерфейс `wlan0`, адрес `192.168.0.50`, контейнер `ml-trainer`, порт `5000`.

Схема обработки: хост А экспортирует набор данных Iris в `raw_data.csv`, очищает его в `clean_data.csv` и передаёт файл по SCP на хост Б; хост Б обучает модель, сохраняет её вместе с проверочной выборкой и по запросу хоста А возвращает предсказания и точность через HTTP API.

Состав проекта и описание скриптов

Проект разделён на два каталога, по одному на каждый узел. В каждом есть Dockerfile, файл `compose.yaml`, шаблон переменных окружения `.env.example` и список зависимостей `requirements.txt`. Оба контейнера после запуска простаивают (`CMD ["sleep", "infinity"]`), а все этапы обработки запускаются вручную командой `docker exec`.

Узел А (каталог node_a)

Образ строится на `python:3.9-slim`, дополнительно устанавливаются `curl` (для запросов к API хоста Б) и `iputils-ping`, из Python-пакетов — `pandas` и `scikit-learn`. Каталог `node_a/data` хоста монтируется в контейнер как `/home/DATA`, поэтому результаты обработки сразу доступны на хосте для передачи по SCP.

```
services:
  data-processor:
    build: .
    container_name: data-processor
    env_file: .env
    volumes:
      - ./data:/home/DATA
    restart: unless-stopped
```

– download_dataset.py — экспортирует встроенный в scikit-learn набор Iris (150 строк, 4 числовых признака) в /home/DATA/raw_data.csv, добавляя целевой столбец species с текстовыми названиями классов. Имя столбца берётся из переменной TARGET_COLUMN. Существующий файл без ключа --overwrite не перезаписывается;

– clean_data.py — читает raw_data.csv и очищает его: у строковых столбцов удаляет пробелы по краям и приводит текст к нижнему регистру, пустые строки заменяет на пропуски, удаляет полностью пустые и неполные строки. Затем проверяет, что данные не опустели, целевой столбец присутствует и содержит не менее двух классов, и сохраняет результат в /home/DATA/clean_data.csv.

Узел Б (каталог node_b)

Образ также строится на python:3.9-slim и содержит Flask, joblib, pandas и scikit-learn. Каталог с данными монтируется только для чтения, артефакты модели сохраняются в отдельный том и переживают пересоздание контейнера; порт 5000 публикуется наружу:

```
services:
  ml-trainer:
    build: .
    container_name: ml-trainer
    env_file: .env
    volumes:
      - ./data:/home/DATA:ro
      - ./artifacts:/home/MODEL
    ports:
      - "5000:5000"
    restart: unless-stopped
```

– train_model.py — читает clean_data.csv и проверяет его: нет пропусков и пустых значений, есть хотя бы один признак, не менее двух классов, в каждом классе не менее двух строк. После этого выполняет стратифицированное разбиение 80/20 (TEST_SIZE=0.2, RANDOM_STATE=42) и обучает конвейер Pipeline: ColumnTransformer пропускает числовые признаки без изменений и кодирует категориальные через OneHotEncoder, классификатор — DecisionTreeClassifier. Модель с метаданными (столбцы, типы, индексы проверочных строк, SHA-256 проверочного файла) сохраняется в /home/MODEL/model.joblib, а проверочная выборка — в /home/MODEL/test_data.csv. Оба файла сначала пишутся во временный каталог и затем атомарно переименовываются, чтобы не оставить несогласованную пару;

– ml_service.py — Flask-приложение на 0.0.0.0:5000. Маршрут /health сообщает, готовы ли модель и тестовые данные. Маршрут /predict загружает

сохранённую модель, сверяет контрольную сумму, столбцы и число строк тестового файла с метаданными, выполняет предсказание и возвращает точность (accuracy) и список пар `row_index`/`prediction`. Если артефактов нет — ответ `503`, если они повреждены или не согласованы — `422`. Модель при запросе не переобучается.

1. Настройка SSH на обоих хостах

На каждом хосте сгенерированы недостающие ключи хоста, в `/etc/ssh/sshd_config` заданы требуемые параметры, конфигурация проверена и служба перезапущена. На Arch Linux служба называется `sshd`:

```
sudo ssh-keygen -A
sudo sed -i -E 's/^#?PermitRootLogin.*/PermitRootLogin no/; s/^#?
MaxAuthTries.*/MaxAuthTries 3/' /etc/ssh/sshd_config
grep -E '^(PermitRootLogin|MaxAuthTries)' /etc/ssh/sshd_config
sudo sshd -t && sudo systemctl enable --now sshd && sudo systemctl restart sshd
sudo sshd -T | grep -Ei 'permitrootlogin|maxauthtries'
systemctl status sshd --no-pager
```

– `ssh-keygen -A` создаёт ключи хоста тех типов, которых ещё нет;

– `sed -i -E` заменяет строки (в том числе закомментированные) на `PermitRootLogin no` — запрет входа под root — и `MaxAuthTries 3` — не более трёх попыток аутентификации за соединение;

– `sshd -t` проверяет синтаксис конфигурации; служба перезапускается только при успешной проверке. `sshd -T` выводит действующие значения параметров и подтверждает, что они применились.

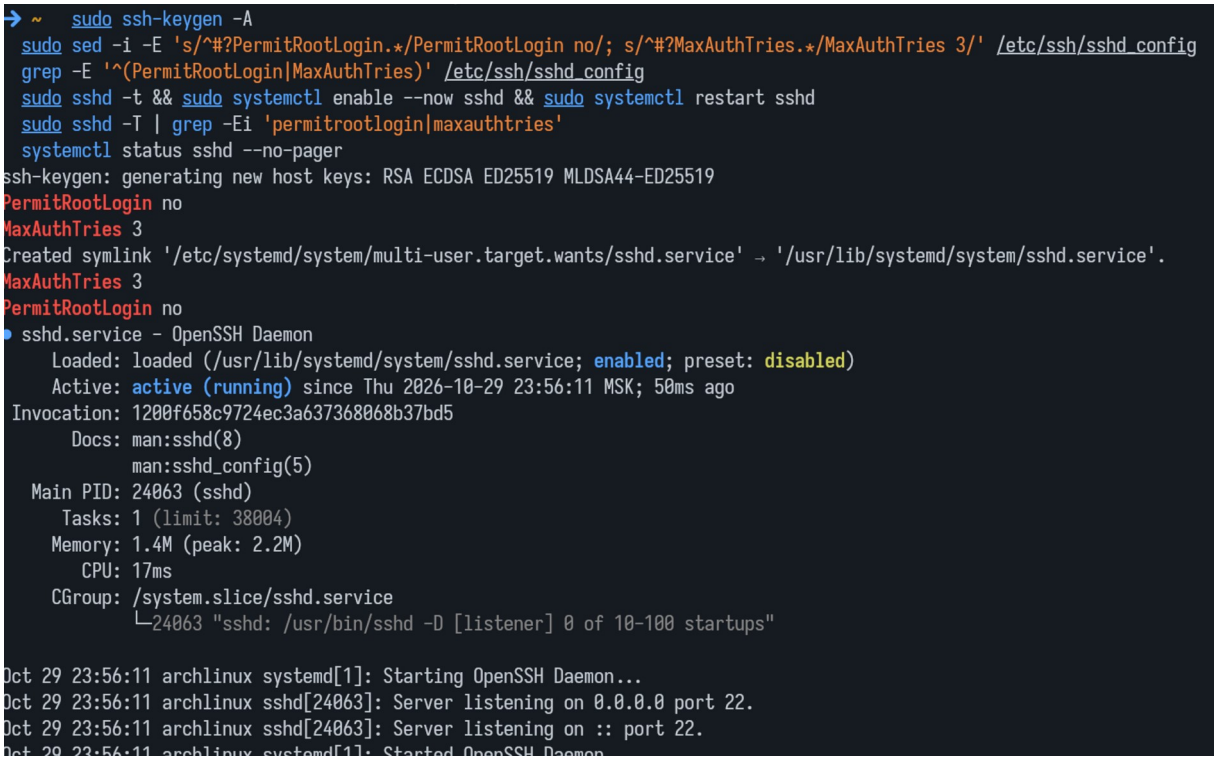


Рисунок 1 — Настройка sshd на хосте A

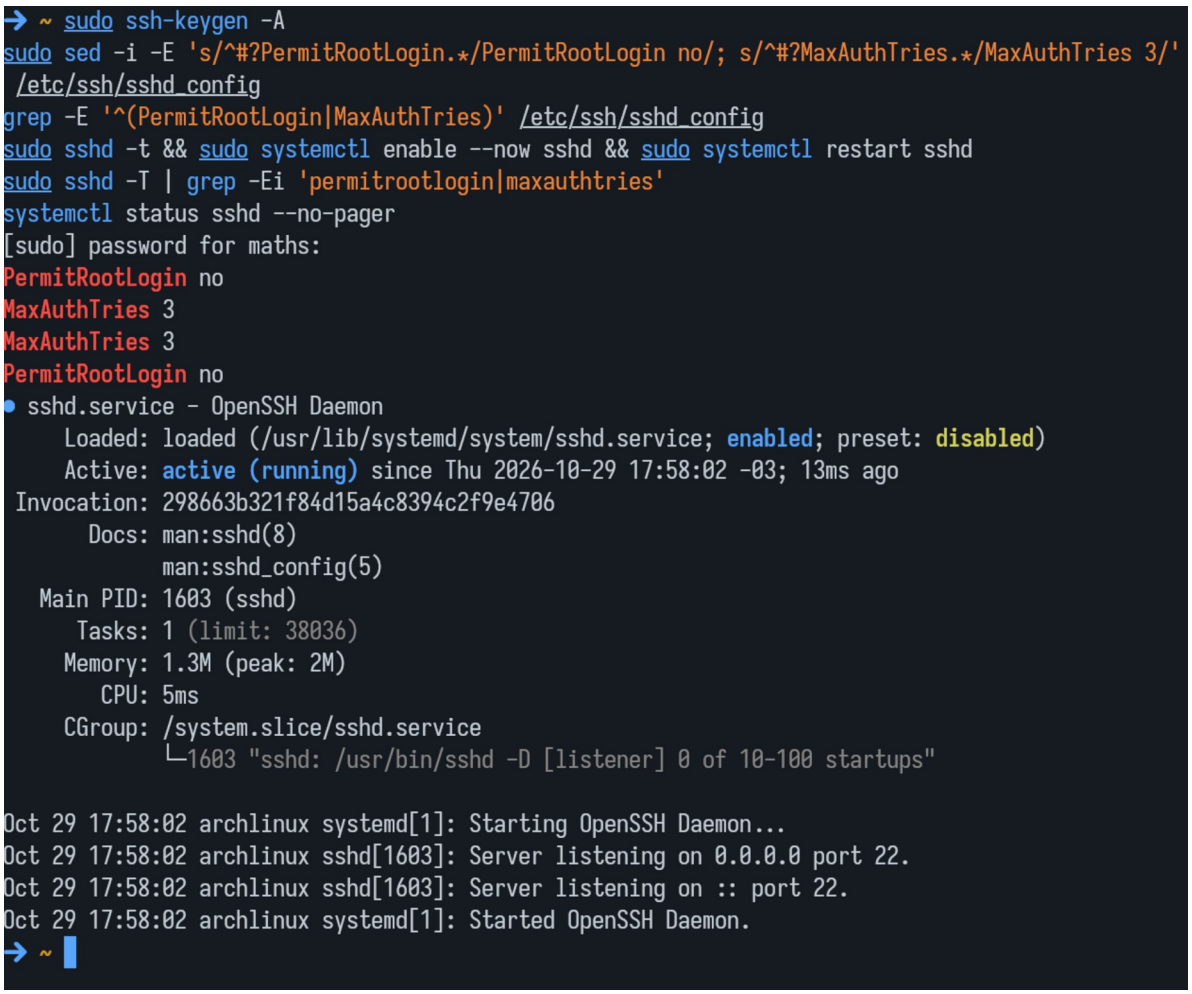


Рисунок 2 — Настройка sshd на хосте Б

На обоих хостах (рисунки 1 и 2) `sshd -T` показывает `PermitRootLogin no` и `MaxAuthTries 3`, служба находится в состоянии `active (running)`, включена в автозапуск (`enabled`) и слушает порт 22. На хосте А ключи хоста ранее отсутствовали и были созданы (RSA, ECDSA, ED25519, MLDSA44-ED25519).

Для входа используется пользовательская пара ключей Ed25519 хоста А, публичный ключ которой уже добавлен пользователю `maths` на хосте Б. Приватный ключ остаётся в `~/.ssh` хоста А и в контейнер не монтируется. Проверка входа по ключу:

```
ssh -i ~/.ssh/id_ed25519 maths@192.168.0.50 'echo OK; whoami'
```

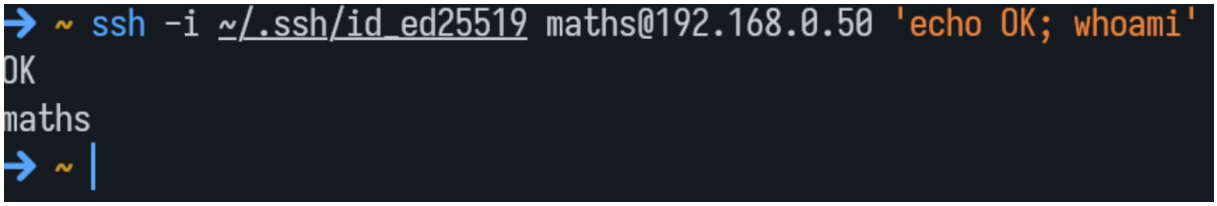


Рисунок 3 — Вход на хост Б по SSH-ключу

Команда выполнилась без запроса пароля и вернула `OK` и имя пользователя `maths` (рисунок 3).

2. Проверка сети и Docker

На каждом хосте выведены сетевые интерфейсы, проверена доступность второго хоста и состояние службы Docker. Для компактности использованы краткий формат `ip -br a` и вывод статуса без журнала (`-n 0`):

```
ip -br a
ping -c 3 192.168.0.50      # на хосте Б: ping -c 3 192.168.0.108
systemctl status docker --no-pager -n 0
```



Рисунок 4 — Сетевые интерфейсы, ping и состояние Docker на хосте А

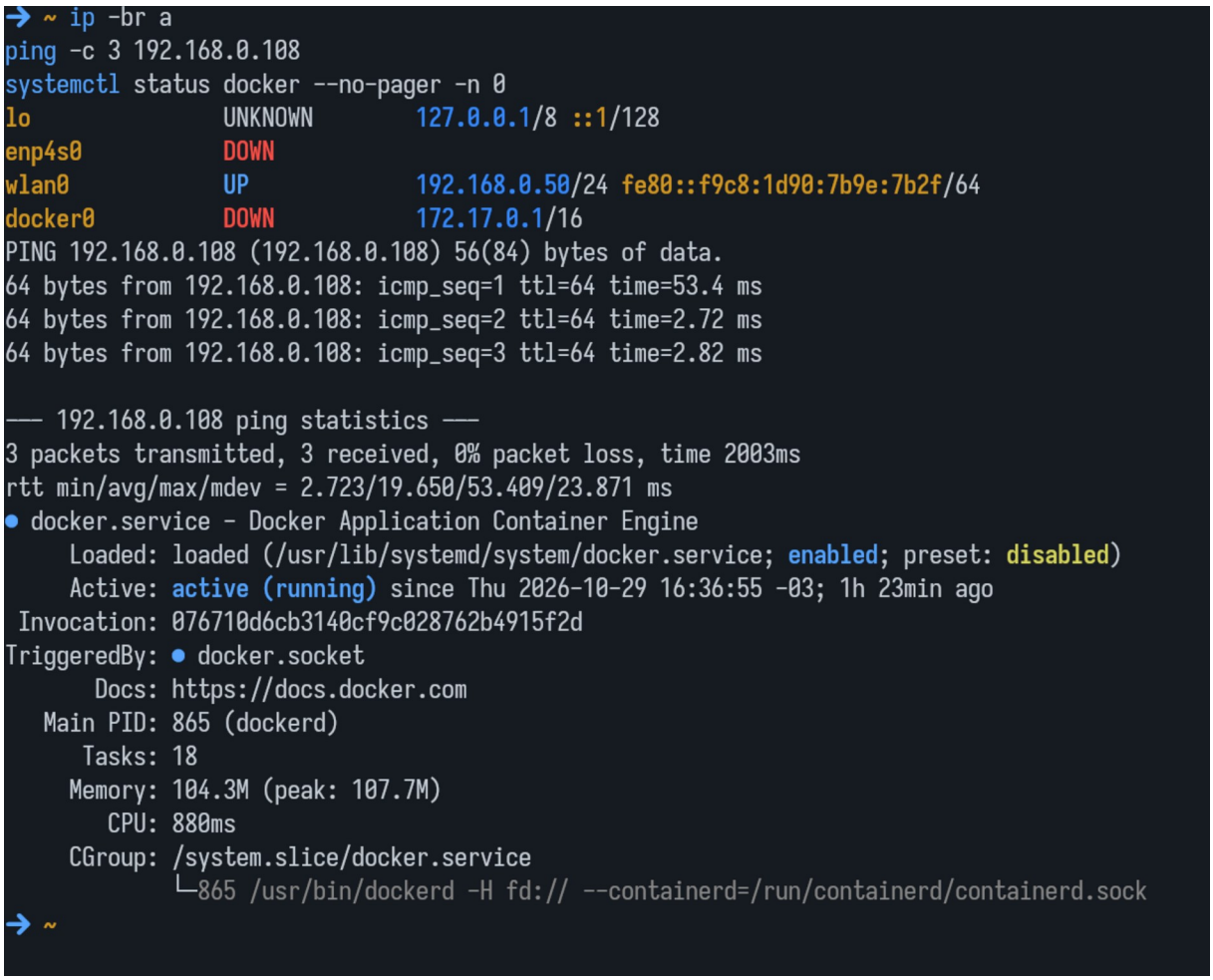


Рисунок 5 — Сетевые интерфейсы, ping и состояние Docker на хосте Б

Хост А подключён через `wlp0s20f3` с адресом `192.168.0.108/24`, хост Б — через `wlan0` с адресом `192.168.0.50/24`. Ping проходит в обе стороны без потерь (3 из 3 пакетов, среднее время около 2,7 мс после первого пакета), служба Docker на обоих хостах активна (рисунки 4 и 5). Ping проверяет только ICMP-связность; доступность TCP-порта Flask проверяется отдельно в разделе 7.

3. Сборка и запуск контейнеров

На каждом хосте из каталога своего узла создан файл `.env` из шаблона, затем образ собран и контейнер запущен в фоне:

```
cd ~/ITMO_AVS_2_lab/node_b          # на хосте А: cd .../ITMO_AVS_2_lab/node_a
cp .env.example .env && cat .env
docker compose up --build -d
docker ps --filter name=ml-trainer  # на хосте А: name=data-processor
realpath data                       # только на хосте Б
```

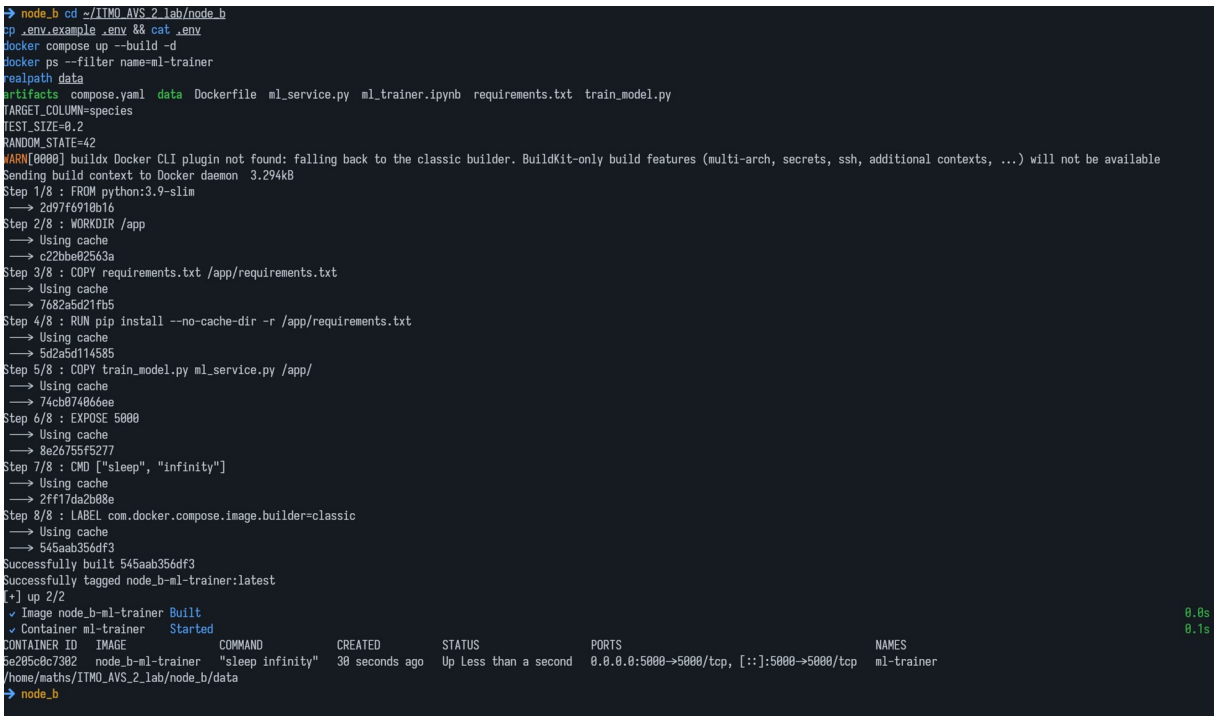


Рисунок 6 — Сборка и запуск контейнера ml-trainer на хосте Б

На хосте Б (рисунок 6) в `.env` заданы `TARGET_COLUMN=species`, `TEST_SIZE=0.2`, `RANDOM_STATE=42`. Плагин buildx на этом хосте не установлен, поэтому Docker выводит предупреждение и собирает образ классическим сборщиком — на результат это не влияет. Контейнер `ml-trainer` запущен, порт 5000 опубликован на всех адресах (`0.0.0.0:5000->5000/tcp`). Команда `realpath data` показала абсолютный путь каталога назначения для SCP: `/home/maths/ITMO_AVS_2_lab/node_b/data`.

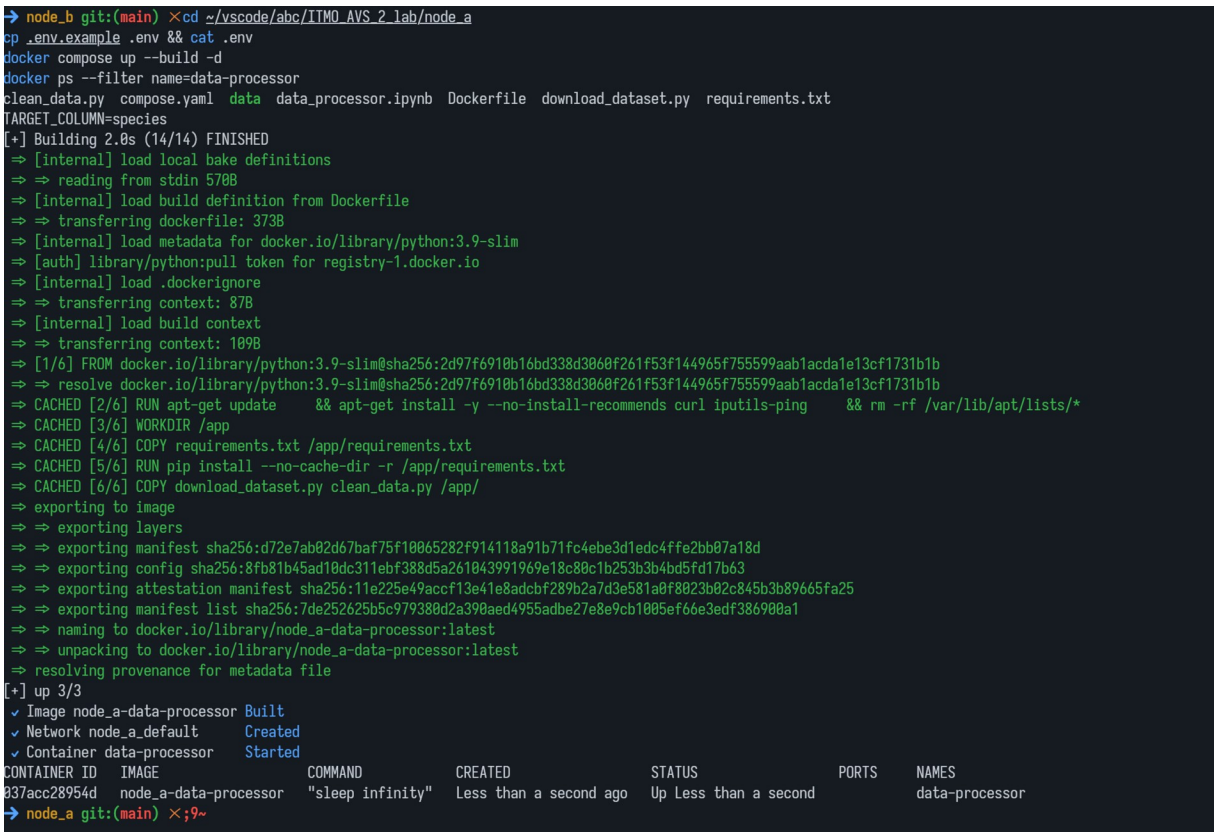


Рисунок 7 — Сборка и запуск контейнера data-processor на хосте A

На хосте A (рисунок 7) в `.env` задан тот же `TARGET_COLUMN=species`, образ `node_a-data-processor` собран BuildKit, контейнер `data-processor` запущен. Оба контейнера выполняют `sleep infinity` и ожидают ручных команд.

4. Подготовка и очистка данных на хосте A

```
docker exec data-processor python /app/download_dataset.py
docker exec data-processor python /app/clean_data.py
head -5 data/raw_data.csv; wc -l data/*.csv
```

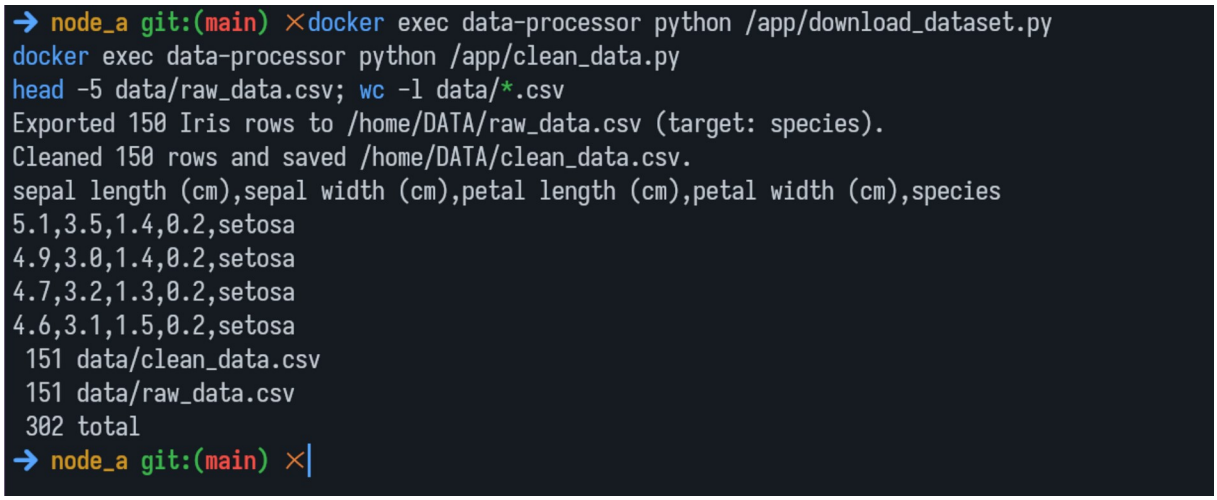


Рисунок 8 — Экспорт набора Iris и очистка данных

Скрипт `download_dataset.py` экспортировал 150 строк Iris в `/home/DATA/raw_data.csv` (это локальный экспорт из scikit-learn, а не сетевое скачивание). Файл содержит четыре числовых признака — длину и ширину чашелистика и лепестка в сантиметрах — и целевой столбец `species`. Скрипт `clean_data.py` сохранил очищенный файл `clean_data.csv`; в наборе Iris нет пропусков, поэтому все 150 строк сохранились. В каждом файле 151 строка: заголовок и 150 записей (рисунок 8). Поскольку каталог `node_a/data` смонтирован в контейнер, оба файла сразу появились на хосте А.

5. Передача очищенного файла на хост Б

Файл передаётся вручную с хоста А по SCP с аутентификацией по ключу, в каталог, путь к которому получен командой `realpath` на хосте Б:

```
scp -i ~/.ssh/id_ed25519 data/clean_data.csv \
    maths@192.168.0.50:/home/maths/ITMO_AVS_2_lab/node_b/data/clean_data.csv
```

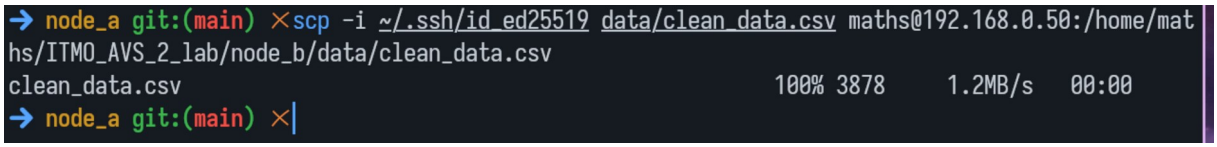


Рисунок 9 — Передача clean_data.csv на хост Б по SCP

Файл размером 3878 байт передан полностью (100 %, рисунок 9). На хосте Б этот каталог смонтирован в контейнер `ml-trainer` как `/home/DATA` только для чтения.

6. Обучение модели на хосте Б

```
cd ~/ITMO_AVS_2_lab
ls -l node_b/data/clean_data.csv
docker exec ml-trainer python /app/train_model.py
ls -l node_b/artifacts
```

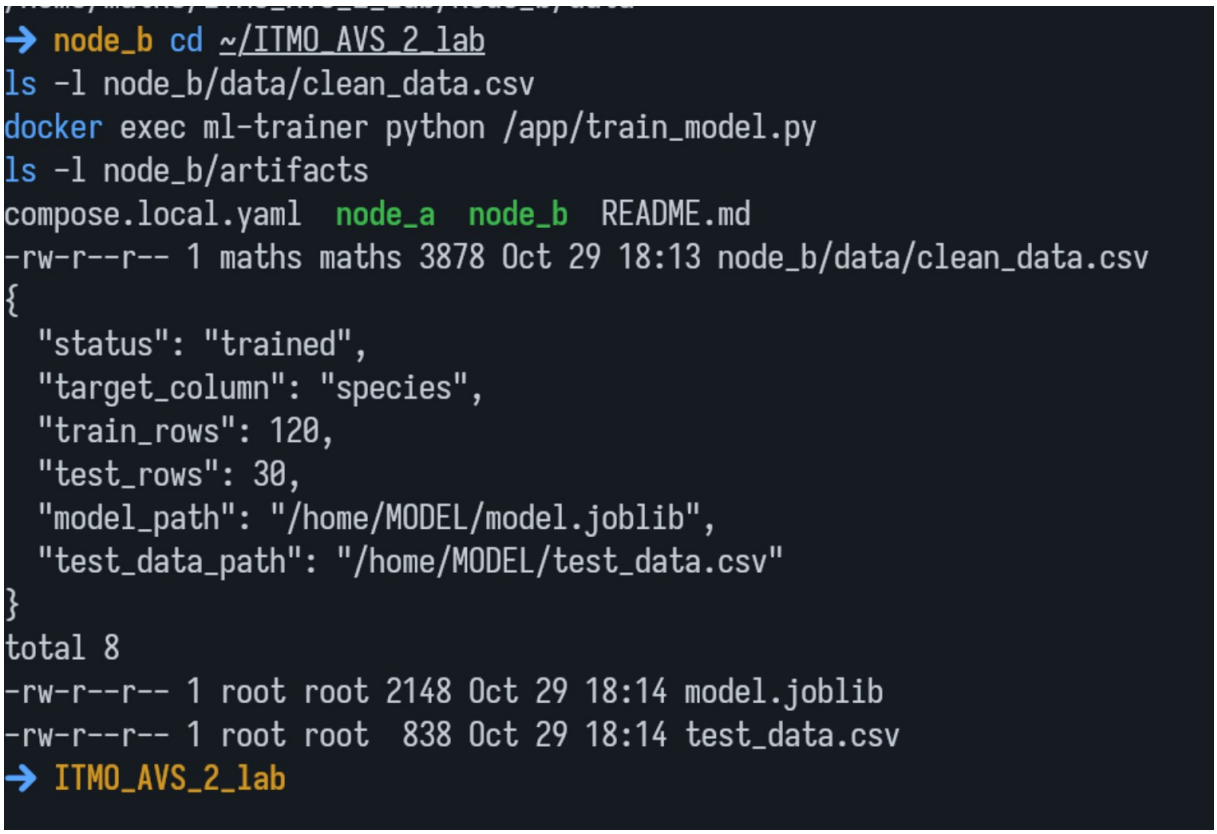


Рисунок 10 — Обучение модели и сохранённые артефакты

Размер полученного файла (3878 байт) совпадает с отправленным. Скрипт обучения вывел сводку в формате JSON (рисунок 10): статус `trained`, целевой столбец `species`, 120 строк обучающей и 30 строк проверочной выборки (разбиение 80/20). В каталоге `node_b/artifacts` появились `model.joblib` (2148 байт) и `test_data.csv` (838 байт). Владелец файлов — `root`, так как процесс в контейнере работает от root.

7. Запуск Flask-сервиса и запрос предсказаний

На хосте Б в отдельном терминале запущен сервер. Он работает на переднем плане и занимает терминал до нажатия Ctrl+C; контейнер при этом продолжает работать:

```
docker exec -it ml-trainer python /app/ml_service.py
```

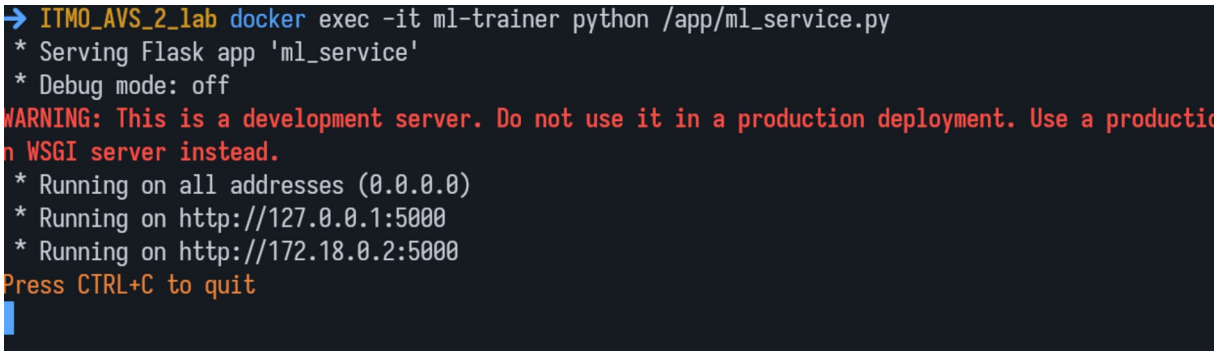


Рисунок 11 — Запуск Flask-сервиса на хосте Б

Сервер слушает все адреса (`0.0.0.0`) на порту 5000; внутри Docker-сети контейнер имеет адрес `172.18.0.2` (рисунок 11). Предупреждение о development-сервере — стандартное сообщение Flask, для лабораторного стенда оно допустимо.

С хоста А запросы выполняются из контейнера `data-processor`:

```
docker exec data-processor curl -s http://192.168.0.50:5000/health
docker exec data-processor curl -s http://192.168.0.50:5000/predict | python -m
json.tool
```

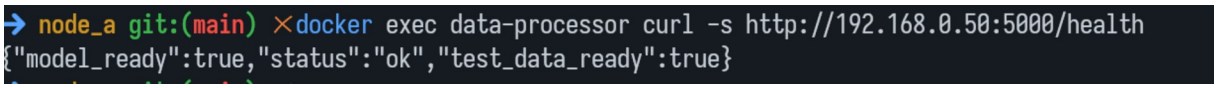


Рисунок 12 — Ответ маршрута /health

Маршрут `/health` вернул `status: "ok"`, `model_ready: true` и `test_data_ready: true` (рисунок 12) — модель и проверочные данные на хосте Б готовы.

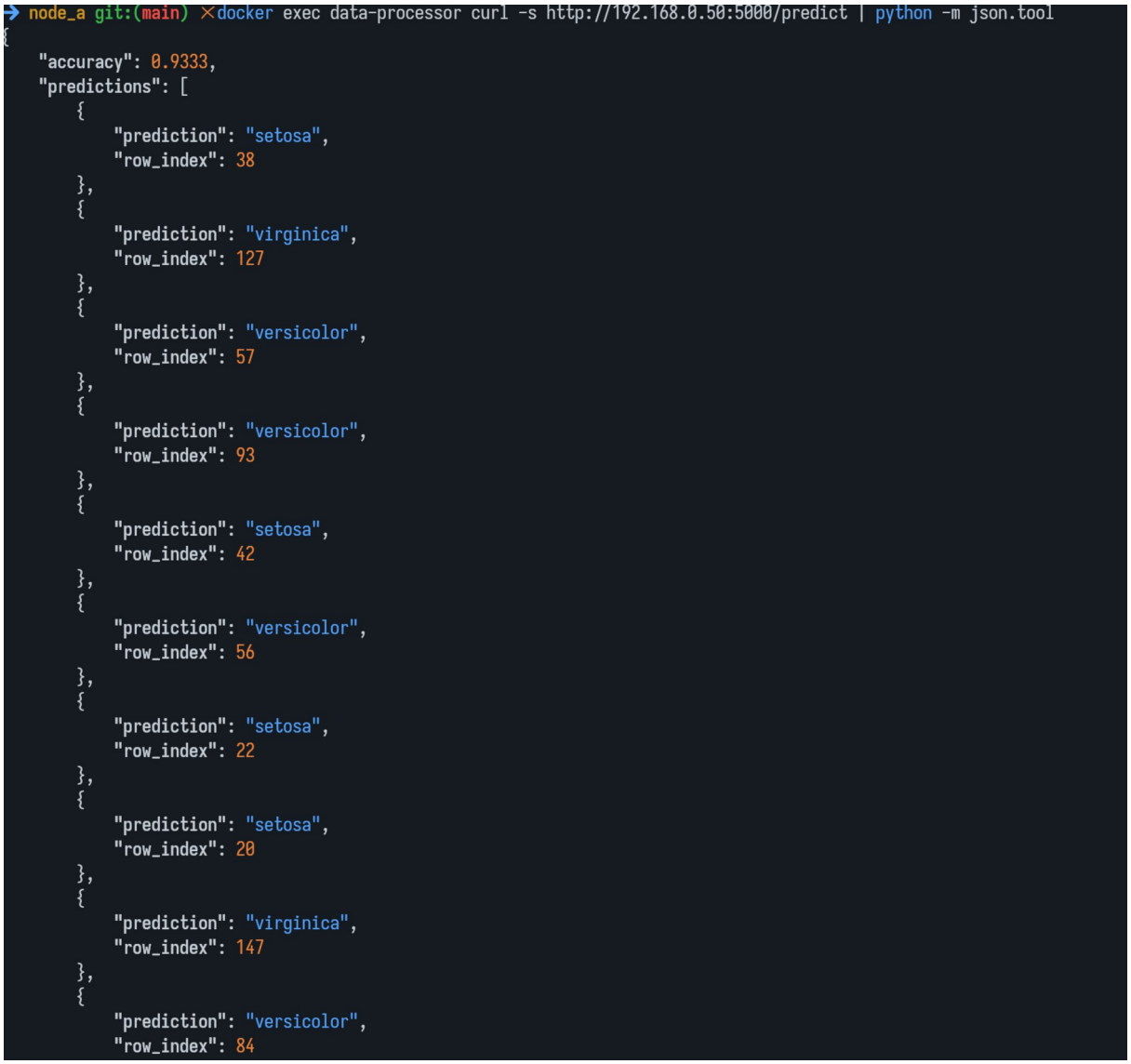


Рисунок 13 — Ответ маршрута /predict (начало)



Рисунок 14 — Ответ маршрута /predict (продолжение и окончание)

Маршрут `/predict` (рисунки 13 и 14) вернул `status: "success"`, `train_rows: 120`, `test_rows: 30`, `target_column: "species"` и массив из 30 предсказаний. Каждый элемент содержит `row_index` — индекс строки в очищенном CSV, начиная с 0, — и предсказанный класс. Точность на проверочной выборке `accuracy = 0.9333`, то есть 28 из 30 объектов классифицированы верно. Предсказание выполняется только по сохранённым артефактам, без повторного обучения и нового разбиения.

8. Ограничение доступа к порту 5000

Docker публикует порт через DNAT, поэтому пакеты к контейнеру проходят цепочку FORWARD, а не INPUT. Пользовательские правила для такого трафика размещаются в цепочке `DOCKER-USER`, которую Docker проверяет первой и не перезаписывает. Сначала выведены текущие правила, затем первым правилом разрешён хост A и вторым запрещены все остальные TCP-соединения к порту 5000:

```
sudo iptables -t nat -nvL
sudo iptables -nvL DOCKER-USER --line-numbers
sudo iptables -I DOCKER-USER 1 -s 192.168.0.108 -p tcp --dport 5000 -j ACCEPT
sudo iptables -I DOCKER-USER 2 -p tcp --dport 5000 -j DROP
sudo iptables -nvL DOCKER-USER --line-numbers
sudo iptables-save | sudo tee /etc/iptables/iptables.rules > /dev/null
```

```
➜ ~ sudo iptables -t nat -nvL
sudo iptables -nvL DOCKER-USER --line-numbers
sudo iptables -I DOCKER-USER 1 -s 192.168.0.108 -p tcp --dport 5000 -j ACCEPT
sudo iptables -I DOCKER-USER 2 -p tcp --dport 5000 -j DROP
sudo iptables -nvL DOCKER-USER --line-numbers
sudo iptables-save | sudo tee /etc/iptables/iptables.rules > /dev/null
[sudo] password for maths:
Chain PREROUTING (policy ACCEPT 4356 packets, 1409K bytes)
 pkts bytes target     prot opt in     out     source               destination
  143 54703 DOCKER     all  --  *      *       0.0.0.0/0            0.0.0.0/0           ADDRTYPE match dst-type LOCAL

Chain INPUT (policy ACCEPT 0 packets, 0 bytes)
 pkts bytes target     prot opt in     out     source               destination

Chain OUTPUT (policy ACCEPT 1761 packets, 110K bytes)
 pkts bytes target     prot opt in     out     source               destination
    1   152 DOCKER     all  --  *      *       0.0.0.0/0           !127.0.0.0/8         ADDRTYPE match dst-type LOCAL

Chain POSTROUTING (policy ACCEPT 1763 packets, 110K bytes)
 pkts bytes target     prot opt in     out     source               destination
    0     0 MASQUERADE  all  --  *      !br-be9ffb2a8da0  172.18.0.0/16       0.0.0.0/0
   12   712 MASQUERADE  all  --  *      !docker0  172.17.0.0/16       0.0.0.0/0

Chain DOCKER (2 references)
 pkts bytes target     prot opt in     out     source               destination
    2   120 DNAT       tcp  --  !br-be9ffb2a8da0 *      0.0.0.0/0            0.0.0.0/0           tcp dpt:5000 to:172.18.0.2:5000
Chain DOCKER-USER (1 references)
num   pkts bytes target     prot opt in     out     source               destination
Chain DOCKER-USER (1 references)
num   pkts bytes target     prot opt in     out     source               destination
1        0     0 ACCEPT     tcp  --  *      *       192.168.0.108        0.0.0.0/0            tcp dpt:5000
2        0     0 DROP       tcp  --  *      *       0.0.0.0/0            0.0.0.0/0            tcp dpt:5000
➜ ~
```

Рисунок 15 — Правила NAT и цепочки DOCKER-USER на хосте Б

В таблице `nat` (рисунок 15) видно правило `DNAT tcp dpt:5000 to:172.18.0.2:5000`, созданное Docker для публикации порта. Цепочка `DOCKER-USER` до изменений была пустой, после — содержит два правила: `1 ACCEPT` для источника `192.168.0.108` и `2 DROP` для всех остальных. Порядок важен: правила проверяются сверху вниз, и разрешающее должно стоять перед запрещающим. В Arch Linux нет пакета `netfilter-persistent`, поэтому правила сохранены командой `iptables-save` в `/etc/iptables/iptables.rules` — этот файл загружает служба `iptables`.

Проверка с разрешённого хоста А:

```
➜ node_a git:(main) ✗docker exec data-processor curl -s http://192.168.0.50:5000/health
{"model_ready":true,"status":"ok","test_data_ready":true}
➜ node_a git:(main) ✗
```

Рисунок 16 — Запрос /health с хоста А после настройки firewall

```
192.168.0.108 - - [29/Oct/2026 21:15:44] "GET /health HTTP/1.1" 200 -
192.168.0.108 - - [29/Oct/2026 21:16:22] "GET /predict HTTP/1.1" 200 -
192.168.0.108 - - [29/Oct/2026 21:19:43] "GET /health HTTP/1.1" 200 -
```

Рисунок 17 — Журнал Flask-сервиса на хосте Б

Хост А по-прежнему получает ответ `status: "ok"` (рисунок 16). В журнале сервера (рисунок 17) видны все запросы с адреса `192.168.0.108`: `/health` и `/predict` до настройки firewall и повторный `/health` после неё — все с кодом 200.

Проверка с постороннего устройства: на телефоне, подключённом к той же сети Wi-Fi, открыт адрес `http://192.168.0.50:5000/health`.

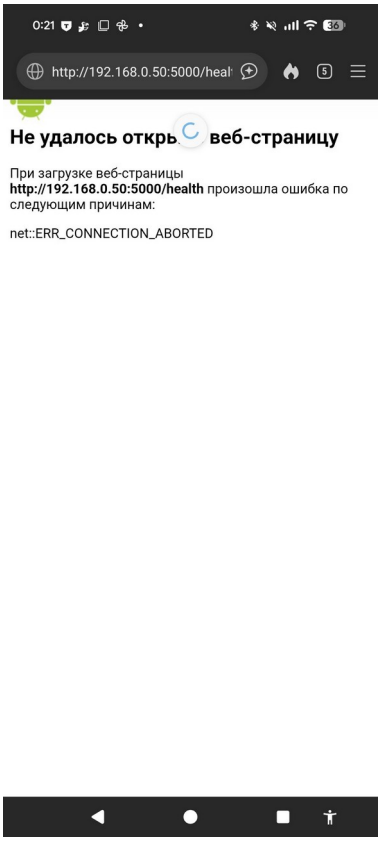


Рисунок 18 — Попытка доступа к сервису с другого устройства

Браузер не смог открыть страницу и сообщил об ошибке `net::ERR_CONNECTION_ABORTED` (рисунок 18), а в журнале сервера запрос с адреса телефона не появился — пакеты отброшены правилом `DROP`. Таким образом, сервис доступен только хосту А.

Вывод

В ходе работы на двух физических хостах под управлением Arch Linux развёрнута распределённая система: узел А подготавливает и очищает данные, узел Б обучает модель классификации и предоставляет HTTP API для получения предсказаний. Модель `DecisionTreeClassifier` на наборе Iris показала точность 0,9333 на 30 проверочных объектах, а доступ к сервису ограничен единственным доверенным хостом.

Были получены следующие знания, умения и навыки:

– настройка SSH-сервера: генерация ключей хоста, запрет входа под root, ограничение числа попыток аутентификации, проверка конфигурации командами `sshd -t` и `sshd -T`;

– аутентификация по ключам и безопасная передача файлов между хостами с помощью `scp`;

– диагностика сети командами `ip`, `ping` и понимание разницы между проверкой ICMP и доступностью TCP-порта;

– описание окружения в Dockerfile и `compose.yaml`: базовый образ, зависимости, переменные окружения из `.env`, тома (в том числе только для чтения), публикация портов, политика перезапуска;

– ручной запуск этапов обработки в работающих контейнерах через `docker exec`;

– очистка табличных данных с помощью pandas, стратифицированное разбиение выборки, построение конвейера scikit-learn и сохранение модели с метаданными через joblib;

– создание REST-сервиса на Flask с маршрутами проверки готовности и инференса и обработкой ошибок через коды HTTP 503 и 422;

– настройка iptables для контейнеров: роль цепочки `DOCKER-USER`, порядок правил ACCEPT/DROP, просмотр таблицы NAT и сохранение правил.

Эти навыки необходимы для развёртывания ML-сервисов в реальной инфраструктуре: передачи данных между серверами, воспроизводимой контейнеризации, публикации моделей через API и защиты сервисов от постороннего доступа.

Использование GAI

Использовалась ли генеративная модель: да.

Модель: Claude Opus 5.5 (компания Anthropic, идентификатор модели `claude-opus-5-5`), использовалась через Claude Code — консольный агент, работающий в терминале хоста A.

Цель использования: только оформление отчёта по лабораторной работе. Модель использовалась строго по чётким инструкциям студента: ей были предоставлены скриншоты выполнения команд, исходные файлы проекта и образец оформления отчёта к работе № 1. Все этапы лабораторной работы на обоих хостах выполнены студентом самостоятельно.